

思想・アプローチ

単なる実装から、大規模開発のための「軌道設計 (Trajectory Design)」へ

学会や海外テック企業が発信する先端の現象や実装パターン。私たちはそれらを単に消費・トレースするのではなく、日本のエンタープライズ・大規模開発が直面する「設計問題」として捉え直し、独自のアーキテクチャへと昇華させてきました。

- **拾う（観察）** Semantic Drift（意味の漂流）、自己回帰に伴う誤差蓄積、長文生成の不安定性、Harness、Plan / Rewind など、先端研究やトップ企業が観測した本質的な課題を深く読み解く。
- **ずらす（再定義）** 評価対象を Wikipedia や単体コーディング環境から、「日本の大規模システム開発、厳格な設計書、成果物、そして人間による承認ゲート」へとシフト。論文の前提を、現場のリアルな制約条件へと再定義する。
- **設計語にする（体系化）** 構造保存、Trajectory Design、Control State、Rollback ≠ Recovery、Portable Agent——。現場で実際にシステムを稼働させ、制御し続けるための「生きた設計語彙」として組み直す。

世界の潮流と、私たちの再定義

先端技術のキーワードを、現場で運用可能なアーキテクチャへと読み替えています。

世界の潮流・先端の議論	私たちが現場向けに組み直した設計思想
Semantic Drift の測定	Softmax Crowding と誤差蓄積 長い開発セッションで文脈が崩壊する構造的要因の解明
Early Stopping / セッション分割	成果物固定と Human Gate プロンプトに依存せず、外部検証とチェックポイントで制御する仕組み
Harness（モデル外の Runtime）	Trajectory Design（軌道設計） どこで生成し、どこで止め、固定し、照合し、引き戻すかの全体設計
Plan / Rewind / Worktree	軌道进行操作するプリミティブ 単なる機能ではなく、AIの思考軌道を制御するための基本要素
ロールバック機能	Rollback ≠ Recovery 単に履歴を戻すことと、正常な制御状態（Control State）への復帰を明確に分離
RAG / ツール呼び出し	Harness 内の Primitive 知識取得は部品に過ぎず、それ単体ではシステムの整合性・軌道は担保できない

技術の系譜：2024 - 2026

私たちはブームに流されることなく、生成AIの「確率的揺らぎ」をいかに工学的に制御するかを一貫して探求してきました。

- **2024年春**：長文生成時の意味の逸脱（Semantic Drift）に着目。
- **2025年前半**：プロンプトブームの中、開発プロセスを「写像（変換）」として捉える設計論を展開。
- **2025年秋～2026年初頭**：海外で Harness Engineering の議論が本格化する中、会話の履歴が制約を薄める構造をモデルの数理から現場向けに言語化。
- **2026年春～現在**：連続生成による構造破壊を防ぐ「成果物固定」や「Trajectory Design」を確立。Harness を軌道制御の工学として体系化。